

JOUR

02

sur 05

DATE DE SESSION

lundi 18 mai 2026

Durée : 4h en présentiel

Le processus de compilation

Programme de la journée

Les 4 étapes de la compilation décortiquées

GCC en pratique : options, drapeaux, fichiers intermédiaires

Bases du C : variables, types, opérateurs, E/S

Atelier — petite calculatrice et lecture des erreurs GCC

Hier vous avez compilé. Aujourd'hui vous allez comprendre.

Ouvrir la boîte noire de GCC

Dernière session

- Différence compilé / interprété
- Histoire et écosystème C/C++
- Installation de GCC
- Premier Hello World en C
- Une commande : `gcc hello.c -o hello`

AUJOURD'HUI

- Décomposer la compilation en 4 étapes
- Voir le code après chaque étape
- Maîtriser les options GCC les plus utiles
- Manipuler variables, types et opérateurs
- Lire les entrées clavier avec `scanf`
- Comprendre les erreurs GCC

Le processus de compilation en 4 étapes

De votre fichier .c à un programme exécutable



Inclut les headers (`#include`), substitue les macros (`#define`), supprime les commentaires.

Analyse syntaxique, génération de code en assembleur (langage proche du processeur).

Traduction de l'assembleur en code machine binaire — fichier objet non encore exécutable.

Le linker relie votre code aux bibliothèques (`printf`, etc.) et produit l'exécutable final.

Étape 1 — Le préprocesseur

Préparer le code avant la vraie compilation

Avant de compiler, GCC exécute un programme appelé `cpp` (C preprocessor) qui réécrit votre code source en suivant les directives qui commencent par `#`.

```
#include <stdio.h>
```

Insère tout le contenu du fichier `stdio.h` à cet endroit

```
#include "mon.h"
```

Idem, mais cherche d'abord dans le dossier courant

```
#define PI 3.14159
```

Définit une constante : toutes les occurrences de `PI` seront remplacées par `3.14159`

```
#define MAX(a,b) ((a)>(b)?(a):(b))
```

Macro avec paramètres — substitution textuelle

```
#ifdef DEBUG / #endif
```

Compilation conditionnelle : le bloc n'est inclus que si `DEBUG` est défini

Voir le résultat du préprocesseur : `gcc -E hello.c -o hello.i`

Étape 2 — La compilation

Du C vers l'assembleur

Le compilateur transforme votre code C (déjà préprocessé) en assembleur — un langage de bas niveau, presque compréhensible par le processeur.

hello.c (C)

```
int main(void) {  
    int a = 5;  
    return a;  
}
```

hello.s (assembleur x86_64)

```
main:  
    pushq %rbp  
    movq  %rsp, %rbp  
    movl  $5, -4(%rbp)  
    movl  -4(%rbp), %eax  
    popq  %rbp  
    ret
```

Voir l'assembleur produit : `gcc -S hello.c -o hello.s`

Étape 3 — L'assemblage

De l'assembleur vers le code machine binaire

L'assembleur (as) prend le fichier .s et produit un fichier objet .o (ou .obj sous Windows).

Le fichier objet contient du code binaire directement compréhensible par le processeur — mais il n'est PAS encore exécutable.

Pourquoi pas encore exécutable ? Parce qu'il fait référence à des fonctions externes (comme printf) dont l'adresse mémoire n'est pas encore connue.

Le format dépend du système ELF sous Linux, Mach-O sous macOS, PE/COFF sous Windows.

Voir un fichier objet Avec `gcc -c hello.c` on obtient `hello.o`, et avec `objdump -d hello.o` on peut désassembler le binaire.

EN COMMANDE

```
gcc -c hello.c  
→ produit hello.o
```

```
file hello.o  
→ ELF 64-bit relocatable
```

```
objdump -d hello.o  
→ désassemblage
```

Étape 4 — L'édition de liens

Le linker assemble tous les morceaux

Rôle du linker (ld) Combine plusieurs fichiers objets et bibliothèques pour produire l'exécutable final.

Résolution des symboles Quand votre code appelle printf, le linker trouve l'implémentation dans la bibliothèque C standard (libc).

Édition statique Le code de la bibliothèque est copié DANS l'exécutable — gros fichier, mais autonome.

Édition dynamique (par défaut) L'exécutable contient seulement des références ; libc est chargée à l'exécution depuis le système.

Erreur fréquente : undefined reference Le linker n'a pas trouvé une fonction. Souvent : oubli d'un .o ou d'une bibliothèque (-lm pour les maths, etc.).

EXEMPLES

Lien avec libm (maths) :
`gcc prog.c -o prog -lm`

Lien de plusieurs fichiers :
`gcc main.o utils.o -o app`

Voir les libs liées :
`ldd ./hello`

GCC en pratique — les options à connaître

Vos drapeaux préférés pour ce module

-o nom	Définit le nom du fichier produit	<code>gcc main.c -o app</code>
-c	Compile uniquement, ne fait pas l'édition de liens (produit un .o)	<code>gcc -c main.c</code>
-Wall	Active TOUS les warnings courants — toujours utilisé en pratique	<code>gcc -Wall main.c</code>
-Wextra	Active des warnings supplémentaires (non couverts par -Wall)	<code>gcc -Wall -Wextra main.c</code>
-g	Inclut les infos de débogage (utile pour gdb au Jour 5)	<code>gcc -g main.c -o app</code>
-O2	Active les optimisations niveau 2 (production)	<code>gcc -O2 main.c -o app</code>
-std=c11	Choisit la version du standard C	<code>gcc -std=c11 main.c</code>

Conseil pédagogique : pendant tout le module, compilez TOUJOURS avec -Wall -Wextra. Les warnings vous évitent des heures de debug.

Variables et types primitifs en C

Le minimum vital pour démarrer

```
main.c — gcc main.c -o main

#include <stdio.h>

int main(void) {
    int    age      = 19;        // entier signé (32 bits)
    float  taille    = 1.78f;    // flottant simple precision
    double pi        = 3.14159;  // flottant double precision
    char   initiale  = 'A';      // un seul caractere (8 bits)
    int    drapeau   = 1;        // pas de bool natif en C89 ; 0 = faux

    printf("Age=%d, Taille=%.2f, Init=%c\n", age, taille, initiale);
    return 0;
}
```

- En C, le type est obligatoire et déterminé à la compilation. Pas de typage dynamique comme en Python.
- Les types courants : int, float, double, char, et leurs variantes signed / unsigned / short / long.
- Les formats de printf (%d, %f, %c, %s) doivent correspondre au type — sinon comportement indéfini !

Tailles des types et plages de valeurs

Sur une machine 64-bit standard

Type	Taille	Plage approximative	Format printf
char	1 octet	-128 à 127	%c (caractère) ou %d (numérique)
unsigned char	1 octet	0 à 255	%u
short	2 octets	-32 768 à 32 767	%hd
int	4 octets	-2.1 milliards à +2.1 milliards	%d
unsigned int	4 octets	0 à 4.29 milliards	%u
long	8 octets	$\approx \pm 9.2 \times 10^{18}$	%ld
float	4 octets	≈ 6 chiffres significatifs	%f
double	8 octets	≈ 15 chiffres significatifs	%lf ou %f

Astuce : utilisez `sizeof(int)` dans votre code pour obtenir la taille réelle sur votre système.

Les opérateurs en C

Calculs, comparaisons, logique

Arithmétiques	Comparaison	Logiques	Affectation
+ addition	== égal	&& ET logique	= affecter
- soustraction	!= différent	 OU logique	+= ajouter et affecter
* multiplication	< inférieur	! NON logique	-= soustraire et affecter
/ division	> supérieur		*= multiplier et affecter
% modulo (reste)	<= inf. ou égal		/= diviser et affecter
++ incrémentation	>= sup. ou égal		

Attention au piège ! La division entière 7 / 2 vaut 3, pas 3.5. Pour avoir un résultat décimal : 7.0 / 2 ou (float)7 / 2.

Entrées / sorties : printf et scanf

Communiquer avec l'utilisateur

```
main.c — gcc main.c -o main

#include <stdio.h>

int main(void) {
    int    age;
    float  taille;
    char   prenom[30];

    printf("Quel est ton prenom ? ");
    scanf("%29s", prenom);

    printf("Quel age as-tu ? ");
    scanf("%d", &age);

    printf("Quelle taille (en m) ? ");
    scanf("%f", &taille);

    printf("Bonjour %s, %d ans, %.2f m.\n", prenom, age, taille);
    return 0;
}
```

- scanf attend l'adresse mémoire d'une variable : on écrit &age (sauf pour les chaînes — un nom de tableau est déjà une adresse).
- Le %29s dans le scanf de la chaîne limite l'entrée à 29 caractères pour éviter le débordement (la chaîne fait 30 cases au total).
- Sans le \n dans printf, le curseur reste sur la même ligne — utile pour des prompts.

Les erreurs en C — 3 grandes familles

Savoir les distinguer pour les corriger vite

Erreurs (errors)

- Le code est syntaxiquement faux — ne compile PAS
- expected ';' before return — point-virgule manquant
- undefined reference to 'printf' — oubli d'inclusion ou de lien
- incompatible types — affectation d'un type vers un autre
- À résoudre AVANT de pouvoir lancer le programme

Avertissements (warnings)

- Le code compile, mais sent mauvais — comportement potentiellement faux
- unused variable 'x' — variable déclarée mais jamais utilisée
- implicit declaration of function — appel d'une fonction non déclarée
- comparison between signed and unsigned — comparaison entre types incompatibles
- À traiter sérieusement : un warning ignoré devient un bug à 3h du matin

Décrypter un message d'erreur GCC

Apprendre à lire de droite à gauche, ligne par ligne

```
$ gcc -Wall main.c -o main
```

Terminal

```
main.c: In function 'main':
```

```
main.c:5:14: error: expected ';' before 'return'
```

```
5 |     int x = 5
```

```
    |                ^
```

```
    |                ;
```

```
6 |     return 0;
```

main.c:5:14

Fichier main.c, ligne 5, colonne 14

error

Erreur (le code ne compilera pas tant qu'elle n'est pas corrigée)

expected ';' before 'return'

Il manque un ; avant le mot return

Le ^

Pointe la position exacte du problème

Compiler proprement — bonnes pratiques

Les habitudes des développeurs C/C++ professionnels

Toujours -Wall -Wextra Le compilateur travaille avec vous, pas contre vous. Activez la vigilance.

Fichiers organisés Un dossier par projet, séparez sources (.c) et headers (.h).

Compiler après chaque modification significative Plus l'erreur est récente, plus elle est facile à trouver.

Lire le PREMIER message d'erreur en priorité Les suivants sont souvent des conséquences en cascade. Corrigez le 1er, recompilez.

Initialiser ses variables Une variable non initialisée contient n'importe quoi (de la mémoire "sale").

Compiler en mode débogage pendant le développement Avec -g pour pouvoir utiliser gdb plus tard.

Mini calculatrice — exemple complet

Toutes les notions du jour réunies

```
main.c — gcc main.c -o main

#include <stdio.h>

int main(void) {
    double a, b;
    char op;

    printf("Calculatrice C\n");
    printf("Operande 1 : ");
    scanf("%lf", &a);
    printf("Operateur (+ - * /) : ");
    scanf(" %c", &op);
    printf("Operande 2 : ");
    scanf("%lf", &b);

    double resultat = 0;
    if (op == '+') resultat = a + b;
    else if (op == '-') resultat = a - b;
    else if (op == '*') resultat = a * b;
    else if (op == '/') resultat = (b != 0) ? a / b : 0;
    else { printf("Operateur inconnu\n"); return 1; }

    printf("Resultat : %.2f %c %.2f = %.2f\n", a, op, b, resultat);
    return 0;
}
```

- L'espace devant %c dans " %c" est important : il consomme les retours-chariot laissés par scanf précédent.
- On vérifie b != 0 avant la division (sinon, division par zéro = comportement indéfini en C).
- Le code retourne 1 en cas d'erreur — convention Unix : 0 = OK, autre = problème.

Atelier 2 — Compilation et calculs

Atelier pratique



MISSION

Manipulez vous-mêmes les étapes de la compilation et écrivez un petit programme interactif.

Étapes à réaliser

1. Compiler hello.c avec gcc -E, -S, -c, puis observez les fichiers .i, .s, .o produits.
2. Compiler le programme avec -Wall -Wextra et vérifier qu'il n'y a aucun warning.
3. Écrire un programme qui demande deux entiers et affiche leur somme, différence, produit, quotient.
4. Provoquer 3 erreurs différentes (oubli de ;, type incompatible, fonction non déclarée) et lire les messages.
5. Bonus : ajouter le calcul du modulo et tester avec des valeurs négatives.



ASTUCE ·

Si vous voyez "undefined reference to sqrt" ou "pow", c'est qu'il faut ajouter -lm à la fin de la commande.

Bilan du Jour 2

Ce que vous savez maintenant faire



ACQUIS DE LA JOURNÉE

- Décrire les 4 étapes : préprocesseur, compilation, assemblage, édition de liens
- Utiliser GCC avec -Wall, -Wextra, -g, -O2, -c, -S, -E
- Déclarer des variables avec les types primitifs et les utiliser
- Lire des données au clavier avec scanf
- Distinguer une erreur de compilation d'un warning
- Lire un message d'erreur GCC et localiser le problème



PROCHAINE ÉTAPE

Demain, on rentre vraiment dans la programmation : les structures de contrôle pour faire des choix (if, switch), répéter des actions (for, while), structurer le code en fonctions, manipuler des tableaux et des chaînes de caractères.

Nous aborderons aussi un concept fondateur du C : les pointeurs.

Révisez bien la slide "opérateurs" : on les utilisera tous demain.